

P1155R1

More implicit moves

Published Proposal, 2018-11-25

Authors:

[Arthur O'Dwyer](#)

[David Stone](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Draft Revision:

9

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d1155-more-implicit-moves.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d1155-more-implicit-moves.html

Abstract

Programmers expect `return x;` to trigger copy elision; or, at worst, to *implicitly move* from `x` instead of copying. Occasionally, C++ violates their expectations and performs an expensive copy anyway. Based on our experience using Clang to diagnose unexpected copies in Chromium, Mozilla, and LibreOffice, we propose to change the standard so that these copies will be replaced with *implicit moves*.

Furthermore, we tentatively propose a new special case to permit efficient codegen for `return x += y.`

Table of Contents

1 Changelog

2 Background

- 2.1 Throwing is pessimized
- 2.2 Non-constructor conversion is pessimized
- 2.3 By-value sinks are pessimized
- 2.4 Slicing is pessimized

3 Proposed wording relative to N4762

4 Proposed wording relative to P0527r1

5 Implementation experience

- 5.1 Plenitude of true positives
- 5.2 Lack of false positives

6 Further proposal to handle assignment operators specially

7 Acknowledgments

References

Informative References

§ 1. Changelog

- R1: Added discussion of `return x += y`.

§ 2. Background

Each version of C++ has improved the efficiency of returning objects by value. By the middle of the last decade, copy elision was reliable (if not technically guaranteed) in situations like this:

```
Widget one() {
    return Widget(); // copy elision
}
Widget two() {
    Widget result;
    return result; // copy elision
}
```

In C++11, a completely new feature was added: a change to overload resolution which I will call *implicit move*. Even when copy elision is impossible, the compiler is sometimes required to *implicitly move* the `return` statement's operand into the result object:

```
std::shared_ptr<Base> three() {
    std::shared_ptr<Base> result;
    return result; // copy elision
}
std::shared_ptr<Base> four() {
    std::shared_ptr<Derived> result;
    return result; // no copy elision, but implicitly moved (not copied)
}
```

The wording for this optimization was amended by [\[CWG1579\]](#). The current wording in [\[class.copy.elision\]/3](#) says:

In the following copy-initialization contexts, a move operation might be used instead of a copy operation:

- If the *expression* in a `return` statement is a (possibly parenthesized) *id-expression* that names an object with automatic storage duration declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression*, or
- if the operand of a *throw-expression* is the name of a non-volatile automatic object (other than a **function** or catch-clause parameter) whose scope does not extend beyond the end of the innermost enclosing *try-block* (if there is one),

overload resolution to select the constructor for the copy is first performed as if the object were designated by an rvalue. If the first overload resolution fails or was not performed, or if the type of the first parameter of the selected **constructor** is not an **rvalue reference** to **the object's** type (possibly cv-qualified), overload resolution is performed again, considering the object as an lvalue.

The highlighted phrases above indicate places where the wording diverges from a naïve programmer's intuition. Consider the following [examples](#)...

§ 2.1. Throwing is pessimized

Throwing is pessimized because of the highlighted word **function** [parameter].

```
void five() {  
    Widget w;  
    throw w; // non-guaranteed copy elision, but implicitly moved (never c  
}  
Widget six(Widget w) {  
    return w; // no copy elision, but implicitly moved (never copied)  
}  
void seven(Widget w) {  
    throw w; // no copy elision, and no implicit move (the object is copie  
}
```

Note: The comment in [seven](#) matches the current Standard wording, and matches the behavior of GCC. Most compilers (Clang 4.0.1+, MSVC 2015+, ICC 16.0.3+) already do this implicit move.

§ 2.2. Non-constructor conversion is pessimized

Non-constructor conversion is pessimized because of the highlighted word **constructor**.

```
struct From {
    From(Widget const &);
    From(Widget&&);
};

struct To {
    operator Widget() const &;
    operator Widget() &&;
};

From eight() {
    Widget w;
    return w; // no copy elision, but implicitly moved (never copied)
}

Widget nine() {
    To t;
    return t; // no copy elision, and no implicit move (the object is copied)
}
```

§ 2.3. By-value sinks are pessimized

By-value sinks are pessimized because of the highlighted phrase **rvalue reference** .

```

struct Fish {
    Fish(Widget const &);
    Fish(Widget&&);
};

struct Fowl {
    Fowl(Widget);
};

Fish ten() {
    Widget w;
    return w; // no copy elision, but implicitly moved (never copied)
}
Fowl eleven() {
    Widget w;
    return w; // no copy elision, and no implicit move (the Widget object
}

```

Note: The comment in `eleven` matches the current Standard wording, and matches the behavior of Clang, ICC, and MSVC. One compiler (GCC 5.1+) already does this implicit move.

§ 2.4. Slicing is pessimized

Slicing is pessimized because of the highlighted phrase **the object's**.

```

std::shared_ptr<Base> twelve() {
    std::shared_ptr<Derived> result;
    return result; // no copy elision, but implicitly moved (never copied)
}
Base thirteen() {
    Derived result;
    return result; // no copy elision, and no implicit move (the object is
}

```

Note: The comment in `thirteen` matches the current Standard wording, and matches the behavior of Clang and MSVC. Some compilers (GCC 8.1+, ICC 18.0.0+) already do this implicit move.

We propose to remove all four of these unnecessary limitations.

§ 3. Proposed wording relative to N4762

Modify [\[class.copy.elision\]/3](#) as follows:

In the following copy-initialization contexts, a move operation might be used instead of a copy operation:

- If the *expression* in a `return` statement is a (possibly parenthesized) *id-expression* that names an object with automatic storage duration declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression*, or
- if the operand of a *throw-expression* is the name of a non-volatile automatic object (other than a ~~function-or~~ catch-clause parameter) whose scope does not extend beyond the end of the innermost enclosing *try-block* (if there is one),

overload resolution to select the constructor for the copy is first performed as if the object were designated by an rvalue. If the first overload resolution fails or was not performed, ~~or if the type of the first parameter of the selected constructor is not an rvalue reference to the object's type (possibly cv-qualified);~~ overload resolution is performed again, considering the object as an lvalue. [Note: This two-stage overload resolution must be performed regardless of whether copy elision will occur. It determines the constructor to be called if elision is not performed, and the selected constructor must be accessible even if the call is elided. —end note]

Note: I believe that the two instances of the word "constructor" in the quoted note remain correct. They refer to the constructor selected to initialize the result object, as the very last step of the conversion sequence. This proposed change merely permits the conversion sequence to be longer than a single step; for example, it might involve a derived-to-base conversion followed by a move-constructor, or a user-defined conversion operator followed by a move-constructor. In either case, as far as the quoted note is concerned, that ultimate move-constructor is the "constructor to be called," and indeed it must be accessible even if elision is performed.

§ 4. Proposed wording relative to P0527r1

David Stone's [\[P0527\]](#) "Implicitly move from rvalue references in return statements" proposes to alter the current rules "references are never implicitly moved-from" and "catch-clause parameters are never implicitly moved-from." It accomplishes this by significantly refactoring clause [\[class.copy.elision\]/3](#).

In the case that [\[P0527\]](#)'s changes are adopted into C++2a, we propose to modify the new [\[class.copy.elision\]/3](#) as follows:

A *movable entity* is a non-volatile object or an rvalue reference to a non-volatile type, in either case with automatic storage duration. ~~The underlying type of a movable entity is the type of the object or the referenced type, respectively.~~ In the following copy-initialization contexts, a move operation might be used instead of a copy operation:

- If the *expression* in a `return` statement is a (possibly parenthesized) *id-expression* that names a movable entity declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression*, or
- if the operand of a *throw-expression* is a (possibly parenthesized) *id-expression* that names a movable entity whose scope does not extend beyond the end of the innermost enclosing *try-block* (if there is one),

overload resolution to select the constructor for the copy is first performed as if the entity were designated by an rvalue. If the first overload resolution fails or was not performed, ~~or if the type of the first parameter of the selected constructor is not an rvalue reference to the (possibly cv-qualified) underlying type of the movable entity,~~ overload resolution is performed again, considering the entity as an lvalue. [Note: This two-stage overload resolution must be performed regardless of whether copy elision will occur. It determines the constructor to be called if elision is not performed, and the selected constructor must be accessible even if the call is elided. —end note]

§ 5. Implementation experience

This feature has effectively already been implemented in Clang since February 2018; see [\[D43322\]](#). Under the diagnostic option `-Wreturn-std-move` (which is enabled as part of `-Wmove`, `-Wmost`, and `-Wall`), the compiler performs overload resolution according to *both* rules — the standard rule and also a rule similar to the one proposed in this proposal. If the two resolutions produce different results, then Clang emits a warning diagnostic explaining that the return value will not be implicitly moved and suggesting that the programmer add an explicit `std::move`.

However, Clang does not diagnose the examples from [§1.3 By-value sinks](#).

§ 5.1. Plenitude of true positives

These warning diagnostics have proven helpful on real code. Many instances have been reported of code that is currently accidentally pessimized, and which would become optimized (with no loss of correctness) if this proposal were adopted:

- [\[SG14\]](#): a clever trick to reduce code duplication by using conversion operators, rather than converting constructors, turned out to cause unnecessary copying in a common use-case.
- [\[Chromium\]](#): a non-standard container library used `iterator::operator const_iterator()` && instead of `const_iterator::const_iterator(iterator&&)`. (The actual committed diff is [here](#).)
- [\[LibreOffice\]](#): "An explicit `std::move` would be needed in the return statements, as there's a conversion from `VclPtrInstance` to base class `VclPtr` involved."

However, we must note that about half of the true positives from the diagnostic are on code like the following example, which is not affected by this proposal:

```
std::string fourteen(std::string&& s) {  
    s += "foo";  
    return s; // no copy elision, and no implicit move (the object is copi  
}
```

See [\[Khronos\]](#), [\[Folly\]](#), and three of the four diffs in [\[Chromium\]](#). [\[AWS\]](#) is a particularly egregious variation. (The committed diff is [here](#).)

```
std::string fifteen() {  
    std::string&& s = "hello world";  
    return s; // no copy elision, and no implicit move (the object is copi  
}
```

Some number of programmers certainly expect a move here, and in fact [\[P0527\]](#) proposes to implicitly move in both of these cases. This paper does not conflict with [\[P0527\]](#), and we provide [an alternative wording](#) for the case that [\[P0527\]](#) is adopted.

§ 5.2. Lack of false positives

In five months we have received a single "false positive" report ([\[Mozilla\]](#)), which complained that the move-constructor suggested by Clang was *not significantly more efficient* than the actually selected copy-constructor. The programmer preferred not to add the suggested `std::move` because the code ugliness was not worth the minor performance gain. This proposal would give Mozilla that minor performance gain without the ugliness — the best of both worlds!

We have never received any report that Clang's suggested move would have been incorrect.

§ 6. Further proposal to handle assignment operators specially

Besides the cases of `return x` handled by this proposal, and the cases of `return x` handled by David Stone's [\[P0527\]](#), there is one more extremely frequent case where a copy is done instead of an implicit move or copy-elision.

```
std::string sixteen(std::string lhs, const std::string& rhs) {  
    return lhs += rhs; // no copy elision, and no implicit move (the object  
}  
  
std::string seventeen(const std::string& lhs, const std::string& rhs) {  
    std::string result = lhs;  
    return result += rhs; // no copy elision, and no implicit move (the object  
}
```

For a real-world example of this kind of code, see GNU libstdc++'s [\[PR85671\]](#), where even a standard library implementor fell into the trap of writing

```
path operator/(const path& lhs, const path& rhs) {  
    path result(lhs);  
    return result /= rhs; // no copy elision, and no implicit move (the object  
}
```

We propose that — in order to make simple code like the above produce optimal codegen — it would be reasonable to create a new special case permitting a (possibly parenthesized) assignment operation to count as "return by name." This would require major surgery on [\[class.copy.elision\]](#). Possibly the best approach would be to introduce a new term, such as "copy-elision candidate," something like this:

When certain criteria are met, an implementation is allowed to omit the copy/move construction of a class object, even if the constructor selected for the copy/move operation and/or the destructor for the object have side effects. Each such case involves an expression, called the *candidate expression*, and a source object, called the *copy elision candidate*.

- In a *return* statement with an *expression*, the candidate expression is the *expression*.
- In a *throw-expression*, the candidate expression is the operand of *throw*.

The copy elision candidate is computed from the candidate expression as follows:

- If the candidate expression is the (possibly parenthesized) name of a non-volatile automatic object, then the copy elision candidate is that object.
- If the candidate expression is an *assignment-expression*, and the *logical-or-expression* on the left-hand side of the *assignment-operator* is the (possibly parenthesized) name of a non-volatile automatic object, and the type of the *assignment-expression* is a non-cv-qualified lvalue reference to the type of the automatic object, then the copy elision candidate is the automatic object.

[Note: This happens regardless of the actual behavior of the assignment operator selected by overload resolution. The implementation essentially assumes that the return value of any (possibly compound) assignment operator is a reference to its left-hand operand. —end note]

The elision of copy/move operations, called copy elision, is permitted in the following circumstances (which may be combined to eliminate multiple copies):

- in a *return* statement in a function with a class return type, when the expression is the name of the copy elision candidate is a non-volatile automatic object (other than a function parameter or a variable introduced by the exception-declaration of a handler (13.3)) with the same type (ignoring cv-qualification) as the function return type, the copy/move operation can be omitted by constructing the automatic object the copy elision candidate object directly into the function call's return object
- in a *throw-expression*, when the operand copy elision candidate is the name of a non-volatile automatic object (other than a function or catch-clause parameter) whose scope does not extend beyond the end of the innermost enclosing try-block (if there is one), the copy/move operation from the operand the copy elision candidate object to the exception object (13.1) can be omitted by constructing the automatic object directly into the exception object

When copy elision occurs, the implementation treats the source and target of the omitted copy/move operation as simply two different ways of referring to the same object. If the first parameter of the selected constructor is an rvalue reference to the object's type, the destruction of

that object occurs when the target would have been destroyed; otherwise, the destruction occurs at the later of the times when the two objects would have been destroyed without the optimization.

This would be a novel special case; as the "Note" says, this would essentially permit the core language to assume that every overloaded `operator=` and `operator@=` which returns an lvalue reference at all, returns an lvalue reference to `*this`. It would be possible for pathological code to observe the optimization happening:

```
struct Observer;
struct Observer {
    static int k = 0;
    static Observer global;
    int i;
    explicit Observer(int i) : i(i) {}
    Observer(const Observer& rhs) : i(++k) {
        printf("observed a copy from %d to %d", rhs.i, i);
    }
    Observer(Observer&& rhs) : i(++k) {
        printf("observed a move from %d to %d", rhs.i, i);
    }
    Observer& operator=(const Observer& rhs) {
        i = rhs.i + 1;
        printf("observed a copy-assign from %d to %d", rhs.i, i);
        return &global; // pathological!
    }
};
Observer Observer::global{10};
Observer foo() {
    Observer x{20};
    Observer y{30};
    return x = y;
}
int main() {
    Observer o = foo();
    printf("o.i is %d\n", o.i);
}
```

In C++17, the above code has this behavior:

- `observed a copy-assign from 30 to 31`, then `observed a copy from 10 to 1`, then `o.i is 1` (the behavior required by C++17, forbidden under the proposal)

Under the "further proposal" sketched above, the code would instead have one of the following behaviors:

- `observed a copy-assign from 30 to 31`, then `observed a move from 10 to 1`, then `o.i is 1` (implicit move, perhaps allowed under the proposal)
- `observed a copy-assign from 30 to 31`, then `o.i is 31` (copy elision, at least allowed and perhaps mandated under the proposal)

§ 7. Acknowledgments

- Thanks to Lukas Bergdoll for his copious feedback.
- Thanks to David Stone for [\[P0527\]](#), and for offering to shepherd P1155R0 at the San Diego WG21 meeting.
- Thanks to Barry Revzin (see [\[Revzin\]](#)) for pointing out the "By-value sinks" case.

§ References

§ Informative References

[AWS]

[Use const references to extend lifetime of temporaries](#). April 2018. URL: <https://github.com/aws/aws-sdk-cpp/issues/847>

[Chromium]

[clean up and enable Wreturn-std-move](#). April 2018. URL: <https://bugs.chromium.org/p/chromium/issues/detail?id=832211>

[CWG1579]

Jeffrey Yasskin. [Return by converting move constructor](#). October 2012. URL: http://open-std.org/JTC1/SC22/WG21/docs/cwg_defects.html#1579

[D43322]

Arthur O'Dwyer. [Diagnose cases of 'return x' that should be 'return std::move\(x\)' for efficiency](#). February 2018. URL: <https://reviews.llvm.org/D43322>

[Folly]

[fix -Wreturn-std-move errors](#). April 2018. URL: <https://github.com/facebook/folly/commit/b5105fc5581eef1af2a809b7a3a50ac820e572ae>

[Khronos]

[Use std::move\(str\) suggested with -Wreturn-std-move](https://github.com/KhronosGroup/SPIRV-Tools/issues/1521). April 2018. URL: <https://github.com/KhronosGroup/SPIRV-Tools/issues/1521>

[LibreOffice]

Stephan Bergmann. [-Werror,-Wreturn-std-move \(recent Clang trunk\)](https://cgit.freedesktop.org/libreoffice/core/commit/?id=74b6e61dde64c5e24bfffacda6f67dbf3d1fc7032). April 2018. URL: <https://cgit.freedesktop.org/libreoffice/core/commit/?id=74b6e61dde64c5e24bfffacda6f67dbf3d1fc7032>

[Mozilla]

[Various '-Wreturn-std-move' build warnings with clang 7.0 \(trunk\), for cases where return invokes \(cheap\) string copy-constructor rather than move constructor](https://bugzilla.mozilla.org/show_bug.cgi?id=1454848). April 2018. URL: https://bugzilla.mozilla.org/show_bug.cgi?id=1454848

[P0527]

David Stone. [Implicitly move from rvalue references in return statements](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0527r1.html). November 2017. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0527r1.html>

[PR85671]

Jonathan Wakely. [Lack of std::move\(\) inside operator/ for std::filesystem::path](https://gcc.gnu.org/bugzilla/show_bug.cgi?id=85671). May 2018. URL: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=85671

[Revzin]

Barry Revzin; Howard Hinnant; Arthur O'Dwyer. [std-proposals thread: By-value sinks](https://groups.google.com/a/isocpp.org/d/msg/std-proposals/eeLS8vI05nM/_BP-8YTPDAAJ). August 2018. URL: https://groups.google.com/a/isocpp.org/d/msg/std-proposals/eeLS8vI05nM/_BP-8YTPDAAJ

[SG14]

Arthur O'Dwyer. [inplace_function implicit conversion chooses copy over move](https://github.com/WG21-SG14/SG14/issues/125). February 2018. URL: <https://github.com/WG21-SG14/SG14/issues/125>